DB Standards

Isin-325

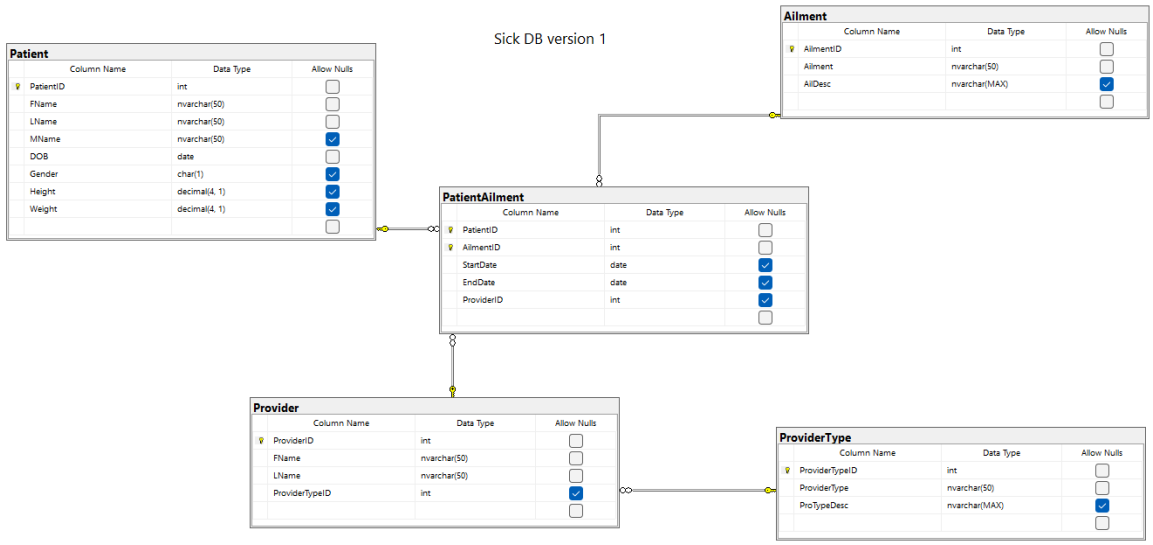
Ethan Krul

4/10/2026

Project Overview

The Sick Database is designed to manage healthcare-related data, including patients, medical conditions, and healthcare providers. The system supports tracking patient diagnoses while maintaining relationships between patients, ailments, and providers. This database follows relational database theory and applies normalization principles to ensure data integrity, reduce redundancy, and prevent update anomalies. The structure separates core entities into individual tables and connects them using primary and foreign keys. The system is designed for Online Transaction Processing, focusing on efficient inserts, updates, and queries for day-to-day healthcare operations.

Database Design Diagram



Design Summary:

- Patient ↔ PatientAilment ↔ Ailment
- Provider ↔ PatientAilment
- Provider ↔ ProviderType

This structure supports:

- Many-to-many relationships (Patient ↔ Ailment)
- Provider tracking per diagnosis
- Scalable and normalized data storage

Data Dictionary

Table: Patient

The Patient table stores demographic and physical information about each patient.

- PatientId (int) is the primary key, not nullable, and indexed. Sample data: 101.
- FName (nvarchar(50)) is not a key, not indexed, and not nullable. Sample data: John.
- LName (nvarchar(50)) is not a key, not indexed, and not nullable. Sample data: Smith.
- MName (nvarchar(50)) is not a key, not indexed, and is nullable. Sample data: A.
- DOB (date) is not a key, not indexed, and not nullable. Sample data: 1995-06-15.
- Gender (char(1)) is not a key, not indexed, and is nullable. Sample data: M.
- Height (decimal(4,1)) is not a key, not indexed, and is nullable. Sample data: 70.5.
- Weight (decimal(4,1)) is not a key, not indexed, and is nullable. Sample data: 180.2.

Table: Ailment

The Ailment table stores medical conditions that can be assigned to patients.

- AilmentId (int) is the primary key, not nullable, and indexed. Sample data: 1.
- Ailment (nvarchar(50)) is not a key, not indexed, and not nullable. Sample data: Diabetes.
- AilDesc (nvarchar(MAX)) is not a key, not indexed, and is nullable. Sample data: Chronic condition affecting blood sugar.

Table: ProviderType

The ProviderType table defines categories of healthcare providers.

- ProviderTypeId (int) is the primary key, not nullable, and indexed. Sample data: 1.
- ProviderType (nvarchar(50)) is not a key, not indexed, and not nullable. Sample data: Physician.
- ProTypeDesc (nvarchar(MAX)) is not a key, not indexed, and is nullable. Sample data: Licensed medical doctor.

Table: Provider

The Provider table stores information about healthcare providers.

- ProviderId (int) is the primary key, not nullable, and indexed. Sample data: 500.

- FName (nvarchar(50)) is not a key, not indexed, and not nullable. Sample data: Sarah.
- LName (nvarchar(50)) is not a key, not indexed, and not nullable. Sample data: Lee.
- ProviderTypeId (int) is a foreign key referencing ProviderType.ProviderTypeId, indexed, and not nullable. Sample data: 1.

Table: PatientAilment

The PatientAilment table is a junction table that links patients, ailments, and providers.

- PatientId (int) is part of the composite primary key, a foreign key referencing Patient.PatientId, not nullable, and indexed. Sample data: 101.
- AilmentId (int) is part of the composite primary key, a foreign key referencing Ailment.AilmentId, not nullable, and indexed. Sample data: 1.
- StartDate (date) is not a key, not indexed, and not nullable. Sample data: 2023-01-01.
- EndDate (date) is not a key, not indexed, and is nullable. Sample data: 2023-06-01.
- ProviderId (int) is a foreign key referencing Provider.ProviderId, indexed, and is nullable. Sample data: 500.

Database Standards Document

Naming Conventions

- Tables and columns use CamelCase
- Primary keys follow format: TableNameId
- Foreign keys match referenced primary key names

Data Types Standards

- int → Used for all primary and foreign keys
- nvarchar(n) → Used for text fields to support Unicode
- nvarchar(MAX) → Used for long descriptions
- date → Used for all date fields
- decimal(4,1) → Used for measurable values like height/weight

Key & Constraint Standards

- All tables must have a primary key

- Foreign keys must enforce referential integrity
- Composite keys allowed only in junction tables
- No orphan records allowed

Normalization Standards

- Database must meet Third Normal Form (3NF)
- No repeating groups (1NF)
- No partial dependencies (2NF)
- No transitive dependencies (3NF)

Indexing Standards

- Primary keys automatically indexed
- Foreign keys indexed for performance
- Avoid excessive indexing in OLTP systems

Null Handling

- Required fields → NOT NULL
- Optional fields → Allow NULL
- Dates like EndDate may be NULL for ongoing treatments

Security & Integrity

- Data integrity enforced through constraints, not application logic
- No duplicate records in junction tables
- Controlled updates through relational constraints

Vendor Standards

- The organization standardizes on Microsoft SQL Server as the primary database management system.
- All database-related tools must be compatible with SQL Server.
- Third-party tools (e.g., reporting, monitoring) must be approved before use to ensure compatibility and supportability.
- Introduction of new database technologies requires management approval and justification.

- To maintain simplicity and reduce overhead, adding new vendors requires evaluation and potential removal of an existing tool.

Platform Standards

- All databases must run on Windows Server environments using SQL Server.
- Production databases must be hosted on dedicated database servers.
- Development and testing environments must be separate from production systems.
- Cross-platform database integration should be minimized to reduce complexity and risk.
- The system follows the KISS principle (Keep It Simple) to maintain manageable infrastructure.

Backup and Recovery Standards

- Full database backups must be performed daily.
- Differential backups must be performed every 6 hours.
- Transaction log backups must be performed every 15 minutes (if applicable).
- Backups must be stored in both local storage and offsite/cloud storage.
- Backup retention period must be at least 30 days.
- Recovery procedures must be tested regularly (at least once per semester or cycle).
- Documentation must exist for all recovery procedures to ensure rapid response during system failure.

Security Standards

- Access to the database must follow the principle of least privilege.
- Role-based access control (RBAC) must be implemented:
    - Admin (full access)
    - Provider (limited medical data access)
    - Read-only users (reporting access only)
- Production and test environments must be strictly separated.
- Sensitive data must only be accessible through controlled queries or stored procedures.
- Authentication must use secure login methods (Windows Authentication preferred).
- All database access and changes must be logged and auditable.

Software Standards

- Applications must access the database using approved queries or stored procedures, not direct table manipulation.
- Naming conventions between database fields and application variables must remain consistent.
- OLTP systems must prioritize fast transactions and data integrity over complex queries.
- Direct database access by end users is prohibited unless explicitly authorized.
- Any software interacting with the database must be tested for performance and security compliance.

Turnover / Change Control Standards

- All database changes must follow a formal approval process before deployment.
- Changes must first be implemented and tested in a development environment.
- A checklist must be used before production deployment, including:
    - Backup verification
    - Script validation
    - Dependency checks
- Changes must be deployed during approved maintenance windows.
- Version control must be maintained for all database scripts and schema changes.
- Any failed deployment must have a rollback plan ready.

Summary of Assignment

This project provided hands-on experience designing a relational database using normalization principles. By structuring the database into separate tables and enforcing relationships through keys, redundancy was minimized and data integrity was maintained. Creating the Data Dictionary helped reinforce understanding of how each column contributes to the overall system and clarified the purpose of data types, constraints, and relationships. Developing database standards emphasized the importance of consistency, scalability, and maintainability in database design.

One of the most valuable aspects of this assignment was identifying and correcting normalization violations. This demonstrated how improper design can lead to data anomalies and how structured design resolves those issues. Overall, this project strengthened my understanding of relational database theory, normalization, and practical database implementation.